Téma 3: Identifikácia používateľov a sedení

📖 Čo sa tu naučím

- **Ako z Apache logu vytvoriť numerický čas (UNIXTIME)** a prečo je nutný pre ďalšie kroky
- **Heuristika IP + User-Agent** na identifikáciu používateľov (~4040 userov)
- **Length** = aproximácia času na stránke cez `groupby + shift`
- **5 metód identifikácie sedení:** STT_MEAN, STT_Q, SLength, RLength, h-ref
- **KS test** na overenie exponenciálneho rozdelenia Length

🔑 Najdôležitejšie vzorce a koncepty

```
UNIX timestamp = pocet sekund od 1.1.1970 00:00 UTC

Length[i] = UNIXTIME[i] - UNIXTIME[i-1]   (v ramci rovnakeho usera)
          = NaN ak Length >= 3600 s

STT_MEAN = mean(Length)
STT_Q    = Q3 + 1.5 * IQR    (Tukeyho pravidlo outliers)
SLength  = 600 s             (10 min, pevny limit na cele sedenie)
RLength  = -ln(1 - p) * mean(Length)   pre p=0.4 (~75 s)
h-ref    = mean(Length) + Referrer == '-'

Exp rozdelenie:  E[X] = 1/lambda,  Median = ln(2)/lambda
KS test: D_n = sup |F_data - F_expon|,  H0: data ~ Exp(lambda)
```

Z prednášky 7 (Fázy procesu Web Log Miningu — identifikácia sedení): Identifikácia používateľov a sedení je kľúčovou súčasťou prípravy dát v metodike CRISP-DM pre Web Usage Mining. Používatelia sa identifikujú **na základe rôznej verzie internetového prehliadača** (kombinácia IP + User-Agent) a sedenia **na základe času, štruktúry prehľadávaných stránok alebo ich kombinácie**. Ak premenná Length (dĺžka návštevy stránky) má exponenciálne rozdelenie, na odhad hraničného času môžeme použiť kvantilovú funkciu `t = -ln(1 - p) / λ`, kde `p` je relatívna početnosť navigačných stránok a `1/λ` je pozorovaný priemer dĺžky návštev (maximálne vierohodný odhad).

Z prednášky 8 (Identifikácia sedení — STT a kvartilový odhad): Ak máme odhad veľkosti časového okna **STT (Session Timeout Threshold)**, tak sedenie je sekvencia navštívených stránok s časovou známkou, pre ktorú platí `t_i - t_{i-1} ≤ STT`. Ak premenná Length **nemá** exponenciálne rozdelenie, na odhad časového okna sa používa **kvartilový odhad** `STT = Q3 + 1.5 · IQR`, kde Q3 je horný kvartil a IQR je kvartilové rozpätie (stredných 50 %

> hodnôt). Odhady strednej hodnoty sa síce používajú, ale ich použitie nie je vhodné (citlivosť na outliers).

Úloha 01: unixtime.py

Teoretický základ: Z prednášky 7 vyplýva, že prvým krokom identifikácie sedení je previesť textový čas Apache logu (`[19/Feb/2020:06:25:49 +0100]`) na numerickú časovú známku. Bez nej nie je možné počítať časové rozdiely medzi po sebe nasledujúcimi záznamami toho istého používateľa, čo je predpoklad pre všetky následné heuristiky (Length, STT, RLength). *(viď prednáška 7 — Fázy procesu, príprava dát)*

Čo robí: Z textového dátumu `'19/Feb/2020:06:25:49 +0100'` vyrobí nový stĺpec UNIXTIME (sekundy od epochy).

Vstup → výstup: - `in/cleaned_log4.csv` (~20666 riadkov) - `out/cleaned_log4_unixtime.csv` (rovnaký + UNIXTIME float)

Kľúčové kroky: 1. `pd.read_csv()` → DataFrame 2. `to_unixtime(dt_str)` parsuje cez `split(':')` a `split('/')` 3. `df['DateTime'].apply(to_unixtime)` → vektorizované na 20k riadkov

Dôležité parametre: - `datetime.datetime(...).timestamp()` — zadanie explicitne pýta (nie pd.to_datetime) - `index=False` pri to_csv — neukladáme RangeIndex - **Hardkódovaný dict mesiacov** — locale-based parsing je nedeterministický

Vzorec / formula:

```
UNIXTIME = sekundy od 1.1.1970 00:00 UTC
```

Časté otázky pri obhajobe: - **Q:** Prečo ručný parser a nie pd.to_datetime? **A:** Zadanie predpisuje `datetime.datetime(...).timestamp()` . pd.to_datetime by bola rýchlejšia (C-vektorizácia), ale nesplnila by literu. - **Q:** Čo s TZ `+0100` ? **A:** Ignoruje sa — vzniká NAIVE datetime, `.timestamp()` ho interpretuje ako lokálny čas systému. Pre CET stroj zhodné s UTC interpretáciou.

Úloha 02: identifikacia_userov.py

Teoretický základ: Podľa prednášky 8 sa používatelia v základnej heuristike identifikujú **na základe rôznej verzie internetového prehliadača** — teda kombinácia `IP + Agent` . Pokročilejšie metódy by používali HTTP/web/Internet/browser **cookie**, no naše dáta cookie neobsahujú. Munk explicitne ukazuje príklad: ten istý záznam IP `91.226.112.32` s rovnakým

Agent `Opera/9.80 (Windows NT 6.1)` patrí jednému používateľovi. *(viď prednáška 8 — Identifikácia používateľov)*

Čo robí: Priradí každej unikátnej dvojici (IP, Agent) sekvenčné USER_ID. **~4040 userov**.

Vstup → výstup: - `in/cleaned_log4_unixtime.csv` - `out/cleaned_log4_users.csv` (pôvodné stĺpce + USER_ID)

Kľúčové kroky: 1. `df.sort_values(by=['IP','Agent','UNIXTIME'])` — chronologicky v rámci usera 2. `df.reset_index(drop=True)` — prečísluje indexy 3. `df['USER_ID'] = df.groupby(['IP','Agent']).ngroup() + 1`

Dôležité parametre: - `drop=True` — nepridáva starý index ako stĺpec - `+1` — z 0-indexed na 1-indexed (človečejšie) - `ngroup()` vs factorize() — ngroup rieši multi-kolumn priamo

Časté otázky pri obhajobe: - **Q:** Limit heuristiky? **A:** Dynamické IP (DHCP) rozdelí usera, zdieľaný počítač zlúči. Bez cookies/login viac neviem. *(viď prednáška 8 — kde sa spomína cookie ako lepšia alternatíva)* - **Q:** Prečo sort pred ngroup? **A:** Cumsum-ová logika v ďalších úlohách predpokladá chronologické poradie + záznamy jedného usera v bloku.

Úloha 03: vypocet_length.py

Teoretický základ: Prednáška 7 zavádza premennú **Length** (dĺžka návštevy stránky) ako rozdiel časových známok dvoch po sebe nasledujúcich prístupov toho istého používateľa. Apache log zaznamenáva iba **príchod** na stránku, nie odchod, preto sa čas strávený na stránke aproximuje rozdielom susedných záznamov. Munk poukazuje, že táto premenná je vstupom pre všetky ďalšie metódy identifikácie sedení a slúži tiež na overovanie exponenciálneho rozdelenia. *(viď prednáška 7 — výpočet Length a maximálne vierohodný odhad parametra λ)*

Čo robí: Spočíta Length = rozdiel UNIXTIME 2 po sebe idúcich prístupov toho istého usera. Priemer ~147 s (očakávaných ~150).

Vstup → výstup: - `in/cleaned_log4_users.csv` - `out/cleaned_log4_length.csv` (+ stĺpec Length, NaN pre prvý záznam usera a Length >= 3600)

Kľúčové kroky: 1. `df['PREV_UNIXTIME'] = df.groupby('USER_ID')['UNIXTIME'].shift(1)` — NaN pre prvý záznam 2. `df['Length'] = df['UNIXTIME'] - df['PREV_UNIXTIME']` 3. `df.loc[df['Length'] >= 3600, 'Length'] = None` — filter > 1 hodina

Dôležité parametre: - `shift(1)` v groupby NEPREKONÁVA hranice skupín - 3600 s = Catledge-Pitkow heuristika (30-60 min koniec sedenia) - `>= 3600` inkluzívne pre None, `< 3600` exkluzívne pre platnú hodnotu

Vzorec / formula:

```
Length[i] = UNIXTIME[i] - UNIXTIME[i-1]   ak rovnaky user
          = NaN                           ak prvy zaznam usera alebo >= 3600 s
```

Časté otázky pri obhajobe: - **Q:** Prečo aproximujeme čas na stránke ako rozdiel? **A:** Apache log zaznamenáva len PRÍCHOD, nie odchod. Predpokladáme, že kliknutie na ďalší link ukončilo čítanie. - **Q:** Prečo NaN pre prvý záznam? **A:** `shift(1)` nevracia nič pre prvý riadok skupiny — to je signál pre ďalšie úlohy ('začiatok usera/sedenia').

Úloha 04: stt_mean.py

Teoretický základ: Prednáška 8 definuje **STT (Session Timeout Threshold)** ako odhad veľkosti časového okna, ktoré rozdeľuje sedenia: ak `t_i - t_{i-1} > STT`, záznamy patria do dvoch rôznych sedení. Najjednoduchším odhadom STT je **priemer Length**, no Munk zdôrazňuje, že použitie strednej hodnoty **nie je vhodné** pre dáta s ťažkým pravým chvostom (typické pre Length). Preto sa zavádzajú robustnejšie odhady (STT_Q, RLength). *(viď prednáška 8 — kvartilový odhad, kritika strednej hodnoty)*

Čo robí: Sedenia metódou STT_MEAN — prah = priemer Length. **~7097 sedení** (očakávaných ~7500).

Vstup → výstup: - `in/cleaned_log4_length.csv` - `out/cleaned_log4_stt_mean.csv` (+ SESSION_ID)

Kľúčové kroky: 1. `stt_mean = df['Length'].mean()` (mean ignoruje NaN automaticky) 2. `df['NEW_SESSION'] = df['Length'].isna() | (df['Length'] >= stt_mean)` 3. `df['SESSION_ID'] = df['NEW_SESSION'].cumsum()` — bool → int

Dôležité parametre: - `.isna()` musí byť OSOBITNE — NaN >= číslo = False v pandas - `|` (bitwise OR), nie `or` — Series nepodporuje pythonovský `or` - `cumsum()` bool: True=1, False=0 → monotónne rastúce ID

Vzorec / formula:

```
NEW_SESSION = Length.isna() OR (Length >= mean(Length))
SESSION_ID = cumsum(NEW_SESSION)
```

Časté otázky pri obhajobe: - **Q:** Prečo priemer? **A:** Hypotéza — ak user na stránke strávil výrazne viac času než priemer, pravdepodobne skončil (káva, zatvoril tab). - **Q:** Nevýhoda? **A:** Priemer je CITLIVÝ na outliers. 1 veľmi dlhá pauza posunie celý prah — STT_Q je robustnejší. *(viď prednáška 8 — „použitie strednej hodnoty nie je vhodné")*

Úloha 05: stt_q.py

Teoretický základ: Prednáška 8 explicitne uvádza: *„Ak premenná Length nemá exponenciálne rozdelenie, na odhad časového okna môžeme použiť kvartilový odhad* `STT_Q = Q3 + 1,5 · IQR`*, kde Q3 je horný kvartil (75. percentil) a IQR je kvartilové rozpätie (stredných 50 % hodnôt).*" Ide o **Tukeyho pravidlo** identifikácie outliers — hodnoty väčšie ako Q3 + 1.5·IQR sa považujú za neprirodzene dlhé pauzy, teda za koniec sedenia. *(viď prednáška 8 — kvartilový odhad STT_Q)*

Čo robí: Sedenia kvartilovým odhadom — **STT_Q = Q3 + 1.5*IQR** (Tukeyho pravidlo). **~7873 sedení**.

Vstup → výstup: - `in/cleaned_log4_length.csv` - `out/cleaned_log4_stt_q.csv` (+ SESSION_ID)

Kľúčové kroky: 1. `q1 = df['Length'].quantile(0.25)`, `q3 = df['Length'].quantile(0.75)` 2. `iqr = q3 - q1`, `stt_q = q3 + 1.5 * iqr` 3. `NEW_SESSION = Length.isna() | (Length >= stt_q)` → `cumsum()`

Dôležité parametre: - `quantile()` ignoruje NaN automaticky (skipna=True default) - `>=` zachytí aj rovnosť (sémanticky OK pre spojité floaty) - Q1 = 25. percentil, Q3 = 75. percentil, IQR = Q3 - Q1

Vzorec / formula:

```
IQR     = Q3 - Q1
STT_Q   = Q3 + 1.5 * IQR
```

Časté otázky pri obhajobe: - **Q:** Prečo kvartily a nie priemer? **A:** ROBUSTNOSŤ — kvartily ignorujú extrémne hodnoty na okrajoch. Length má ťažký pravý chvost. - **Q:** Čo je Tukeyho pravidlo? **A:** Hodnota > Q3 + 1.5*IQR sa považuje za outlier. Pre Length to znamená 'neprirodzene dlhá pauza' → nové sedenie.

Úloha 06: slength.py

Teoretický základ: Prednáška 8 odlišuje dva typy odhadov: **STT** (časové okno medzi po sebe nasledujúcimi záznamami, `t_i - t_{i-1} ≤ STT`) a **odhad dĺžky času sedenia** (celé sedenie nesmie prekročiť limit `t_n - t_1 ≤ Length_session`). SLength patrí do druhej kategórie — pevný 10-minútový limit na **celú dĺžku sedenia**, nie na jednotlivé prechody. *(viď prednáška 8 — „Ak máme odhad dĺžky času sedenia, tak sedenie je sekvencia ... pre ktoré platí t_n - t_1 ≤ Length")*

Čo robí: Sedenia pevným 10 min limitom na **CELÚ dĺžku sedenia** (nie čas medzi prístupmi). **~6700 sedení**.

Vstup → výstup: - `in/cleaned_log4_length.csv` - `out/cleaned_log4_slength.csv` (+ SESSION_ID)

Kľúčové kroky: 1. `slength = 600` (10 min) 2. Iteruj cez `iterrows()`, kumuluj Length 3. Ak `Length is NaN` ALEBO `cumulative + Length >= 600` → nové sedenie + reset kumulatívu

Dôležité parametre: - `iterrows()` nie cumsum — kumulatív sa musí RESETOVAŤ pri prahu - Pri prahu: `cumulative_time = row['Length']` (Length sa stane prvým príspevkom nového sedenia) - `pd.isna()` NaN-safe (NaN == NaN by vrátilo False)

Vzorec / formula:

```
sum_Length(session) < 600 s  → pokracovanie
sum_Length(session) + Length[i] >= 600 s → nove sedenie od i
```

Časté otázky pri obhajobe: - **Q:** Čo je rozdiel od STT? **A:** STT delí podľa času MEDZI prístupmi. SLength podľa CELKOVEJ dĺžky sedenia. *(viď prednáška 8 — dve rôzne definície sedenia)* - **Q:** Prečo `iterrows()` a nie vektorizácia? **A:** `cumsum()` je monotónny — nevie sa vynulovať pri dosiahnutí prahu. Pri 20k riadkoch je iterrows réžia zanedbateľná.

Úloha 07: ks_test_exp_rozdelenia.py

Teoretický základ: Prednáška 7 zavádza predpoklad, že premenná Length má **exponenciálne rozdelenie** `f(x; λ) = λ·e^{-λx}`, `F(x) = 1 - e^{-λx}`, kde `λ > 0`. Maximálne vierohodný odhad parametra `λ` je `λ̂ = 1/x̄`, kde `x̄` je pozorovaný priemer dĺžky návštev. Tento predpoklad je nutný pre platnosť RLength heuristiky — pred jej použitím treba **overiť hypotézu** o exponenciálnom rozdelení (KS test). *(viď prednáška 7 — Exp rozdelenie + MLE λ)*

Čo robí: Overuje hypotézu **Length ~ Exp(lambda)** cez Kolmogorov-Smirnov test + vizualizácia.

Vstup → výstup: - `in/cleaned_log4_length.csv` - `out/rlength_exp.png` (histogram + QQ plot) - stdout: KS štatistika, p-hodnota, rozhodnutie

Kľúčové kroky: 1. `length_data = df['Length'].dropna()` 2. `kstest(length_data, 'expon', args=(0, mean))` — odhad scale=mean 3. Histogram s `expon.pdf(x, 0, mean)` + QQ plot s `expon.ppf((i+0.5)/n, 0, mean)`

Dôležité parametre: - `args=(0, length_data.mean())` — loc=0 (Length >= 0), scale=mean (1/lambda MLE odhad) - `density=True` v hist — normalizuje plochu na 1 (porovnateľné s PDF) - alpha=0.05 — hladina významnosti

Vzorec / formula:

```
Exp PDF:   f(x; lambda) = lambda * exp(-lambda * x),  x >= 0
E[X]       = 1/lambda
Median     = ln(2)/lambda ~ 0.693 * mean
```

```
KS:         D_n = sup_x |F_data(x) - F_expon(x)|
H0:         F_data = F_expon  →  ak p < 0.05, zamietame
```

Časté otázky pri obhajobe: - **Q:** Prečo môže byť p-hodnota anti-konzervatívna? **A:** Lebo scale odhadujeme z dát. Lilliefors / bootstrap KS by boli korektnejšie. - **Q:** Čo je QQ plot? **A:** Body ležia na y=x ak data ~ Exp. Nad čiarou = ťažšie chvosty, pod čiarou = ľahšie. - **Q:** Prečo vlastne overujeme exponencialitu? **A:** Pretože RLength vzorec `t = -ln(1-p) · mean(Length)` vychádza z inverznej CDF Exp rozdelenia — ak Length nie je Exp, použiť RLength nemá teoretický zmysel a treba prejsť na STT_Q. *(viď prednáška 8 — „Ak premenná Length nemá exponenciálne rozdelenie, môžeme použiť kvartilový odhad")*

Úloha 08: rlength.py

Teoretický základ: Prednáška 7 odvodzuje vzorec RLength z **kvantilovej funkcie exponenciálneho rozdelenia**: ak `F(t) = 1 - e^{-λt} = p`, tak `t = F^{-1}(p) = -ln(1-p)/λ`. Munk uvádza: *„Ak je `μ` relatívna početnosť navigačných stránok, na odhad hraničného času τ môžeme použiť kvantilovú funkciu τ = -ln(1-μ)/λ pre 0 ≤ μ < 1."* Pre projekt sa štandardne berie `μ = 0.4` (40 % stránok je navigačných). *(viď prednáška 7 — kvantilová funkcia Exp rozdelenia)*

Čo robí: Sedenia **RLength** heuristikou — prah z kvantilovej funkcie Exp rozdelenia pre p=40 % navigačných stránok. ~75 s prah → **~7558 sedení**.

Vstup → výstup: - `in/cleaned_log4_length.csv` - `out/cleaned_log4_rlength.csv` (+ SESSION_ID)

Kľúčové kroky: 1. `p = 0.4`, `mean_length = df['Length'].mean()` 2. `rlength = -np.log(1 - p) * mean_length` (~0.5108 * mean) 3. `NEW_SESSION = Length.isna() | (Length >= rlength)` → `cumsum()`

Dôležité parametre: - `np.log` = PRIRODZENÝ logaritmus (ln). Nie np.log10/np.log2. - p = 0.4 = relatívna početnosť navigačných stránok - Prah ~75 s vs STT_MEAN ~147 s → VIAC sedení

Vzorec / formula:

```
Exp CDF:  F(t) = 1 - exp(-t/mean) = p
=>        t = -ln(1 - p) * mean(Length)
```

Časté otázky pri obhajobe: - **Q:** Aký je predpoklad RLength? **A:** Length má EXPONENCIÁLNE rozdelenie (overujeme cv07 KS testom). *(viď prednáška 7)* - **Q:** Prečo viac sedení ako STT_MEAN? **A:** Prah je nižší (~75 s vs ~147 s), takže častejšie citlivejšie delenie. - **Q:** Čo je p=0.4? **A:** 40 % stránok je navigačných (user klikol a šiel ďalej). 60 % číta obsah → možné ukončenie sedenia.

Úloha 09: h_ref.py

Teoretický základ: Prednáška 8 zavádza tretiu rodinu metód: **identifikácia sedení na základe kombinácie času a štruktúry** (Extended Log File, ELF). Sedenie je sekvencia záznamov, pre ktorú platí `Referrer_i = URL_{i-1}` — teda nasledujúca stránka musí byť odkazovaná z predchádzajúcej. Ak Referrer nezodpovedá (alebo `Referrer = '-'`, t.j. externý/priamy prístup), záznamy patria do dvoch rôznych sedení. h-ref kombinuje STT_MEAN s touto štrukturálnou podmienkou. *(viď prednáška 8 — identifikácia sedení na základe štruktúry + kombinácia času a štruktúry)*

Čo robí: Sedenia **kombináciou čas + štruktúra**: STT_MEAN prah + `Referrer == "-"`. **~7200 sedení**.

Vstup → výstup: - `in/cleaned_log4_length.csv` - `out/cleaned_log4_href.csv` (+ SESSION_ID)

Kľúčové kroky: 1. `mean_length = df['Length'].mean()` 2. `NEW_SESSION = Length.isna() | (Length >= mean_length) | (Referrer == '-')` 3. `SESSION_ID = NEW_SESSION.cumsum()`

Dôležité parametre: - 3 podmienky cez `|` (bitwise OR) - `Referrer == '-'` = HTTP hlavička Referer chýba (priama URL/záložka/externý odkaz)

Vzorec / formula:

```
NEW_SESSION = isna(Length) OR (Length >= mean) OR (Referrer == '-')
```

Časté otázky pri obhajobe: - **Q:** Čo rozširuje h-ref oproti STT? **A:** Pridáva štrukturálnu heuristiku — externý/priamy prístup je jasný signál pre nové sedenie, nezávisle od času. - **Q:** Čo znamená Referrer `-`? **A:** Apache zapisuje `-` keď Referer hlavička chýba — priame zadanie URL, záložka, externý link.

🎯 Vyhodnotenie / na čo si dať pozor pri obhajobe

- **Všetky session metódy zdieľajú cumsum boolean trick** — preto je dôležité, že prvý záznam usera má Length=NaN (z `shift` v úlohe 03). Bez toho by sa sedenia rôznych userov zlučovali.
- **Munk často pýta** prečo `|` a nie `or` v pandas — Series nepodporuje pythonovský 'or', vyhodí ValueError 'ambiguous truth value'.
- **Robustnosť** — STT_MEAN je nevhodné pre Length kvôli ťažkému pravému chvostu. STT_Q je robustný. *(viď prednáška 8 — „použitie strednej hodnoty nie je vhodné")*
- **Počty sedení v tolerancii** — očakávania zo zadania sú 'cca' ± 5–10 %.

- **KS test anti-konzervatívny** — odhadujeme scale z dát, takže p-hodnota je vyššia ako reálna. Korektnejší: Lilliefors / bootstrap.
- **Všetky skripty zapisujú s** `index=False` — inak by sa vytvoril stĺpec 'Unnamed: 0' pri každom ďalšom `read_csv`.
- **Vzorec RLength**: pozná sa naspamäť — `t = -ln(1-p) * mean(Length)`, p = 40 % navigačných. Vychádza z `F^-1(p)` inverznej CDF Exp. *(viď prednáška 7)*
- **Vzorec STT_Q**: `Q3 + 1.5 * IQR` — Tukeyho pravidlo outliers. *(viď prednáška 8)*
- **Tri rodiny metód v prednáške 8**: (1) podľa štruktúry — Referrer, (2) podľa dĺžky sedenia — SLength, (3) podľa časového okna STT — STT_MEAN, STT_Q, RLength. h-ref je **kombinácia** (1) + (3).